

P1L0 - Ejemplo Minimo 2D OpenSeesPy

Marco isostatico de tres articulaciones
Pregunta 2 - Control 1 Estructuras Isostaticas

Grupo: MCOC-grupo1
Curso: Analisis Estructural
Fecha de entrega: Jueves 20 de agosto 2026

Documento generado con ayuda de OpenCode (IA)

1. QUE MODELAMOS

Ejercicio

Se modela el marco isostatico de la Pregunta 2 del Control 1 de Estructuras Isostaticas. El marco tiene 4 barras inclinadas (AB, BC, CD, DE), apoyos articulados en A y E, y una rotula interna en C. Se le aplica una carga distribuida vertical de 3 tonf/m sobre la proyeccion horizontal.

Geometria

A = (0.0, 0.0) m E = (13.0, 0.0) m
B = (4.0, 3.0) m
C = (6.5, 3.0) m (punto de rotula interna)
D = (9.0, 3.0) m

Seccion: HE 340 AA (acero)

E = 200 GPa = 200e9 Pa
A = 9424.5 mm²
Iz = 182805143.4 mm⁴
Wz = 1142532.1 mm³
tw = 8.5 mm

Carga

q = 3 tonf/m (vertical, hacia abajo, sobre proyeccion horizontal)

Apoyos

- Nodo A: apoyo articulado (restringe ux y uy, libre rz)
- Nodo E: apoyo articulado (restringe ux y uy, libre rz)
- Nodo C: rotula interna (equalDOF comparte ux, uy pero no rz)

Tipo de modelo OpenSees

```
ops.model("basic", "-ndm", 2, "-ndf", 3)
# 2 dimensiones, 3 GDL por nodo: ux, uy, rz
```

2. IMPORTACIONES Y CONSTANTES

Librerias

```
from __future__ import annotations # Tipos modernos
import math                        # hypot, isclose
from pathlib import Path           # Rutas de archivos
import matplotlib                  # Graficos
matplotlib.use("Agg")              # Sin pantalla
import matplotlib.pyplot as plt    # Interfaz de graficos
import openseespy.opensees as ops # Motor de analisis
```

Constantes de conversion

```
TONF    = 1_000.0 * 9.8    # 1 tonf = 9800 N
MM2_TO_M2 = 1.0e-6        # mm2 a m2
MM4_TO_M4 = 1.0e-12       # mm4 a m4
MM3_TO_M3 = 1.0e-9        # mm3 a m3
MPA       = 1.0e6         # 1 MPa = 1e6 Pa
```

Razon: el enunciado entrega propiedades en mm y tonf, pero OpenSees trabaja en SI (m, N, Pa). Hay que convertir todo.

Funciones auxiliares de conversion

```
def tonf(value_newton):
    return value_newton / TONF

def tonf_m(value_newton_meter):
    return value_newton_meter / TONF
```

3. FUNCION: CARGA DISTRIBUIDA

Esta es la funcion mas importante del codigo. Resuelve un problema real: el enunciado dice '3 tonf/m sobre la proyeccion horizontal', pero OpenSees aplica cargas en ejes LOCALES del elemento. Los elementos AB, BC, CD, DE estan inclinados, entonces hay que convertir.

Paso 1: Obtener geometria del elemento

```
xi, yi = ops.nodeCoord(node_i) # Coordenadas nodo inicial
xj, yj = ops.nodeCoord(node_j) # Coordenadas nodo final
dx = xj - xi                    # Distancia horizontal
dy = yj - yi                    # Distancia vertical
length = math.hypot(dx, dy)    # Longitud real del elemento
cos_theta = dx / length        # Coseno del angulo
sin_theta = dy / length        # Seno del angulo
```

Paso 2: Calcular carga en longitud real

```
q_on_member = q_horizontal * abs(dx) / length
# La carga por metro horizontal se distribuye sobre la
# longitud real inclinada del elemento
```

Paso 3: Carga global (vertical hacia abajo)

```
load_global_x = 0.0            # No hay carga horizontal
load_global_y = -q_on_member    # Negativo = hacia abajo
```

Paso 4: Rotacion global -> local

```
load_local_x = load_global_x * cos_theta + load_global_y * sin_theta
load_local_y = -load_global_x * sin_theta + load_global_y * cos_theta
```

Matriz de rotacion 2D:

$$\begin{bmatrix} F_x' \\ F_y' \end{bmatrix} = \begin{bmatrix} \cos & \sin \\ -\sin & \cos \end{bmatrix} \begin{bmatrix} F_x \\ F_y \end{bmatrix}$$

Esto convierte fuerzas de coordenadas globales a locales del elemento.

Paso 5: Aplicar en OpenSees

```
ops.eleLoad("-ele", ele_tag, "-type", "-beamUniform",
            load_local_y, load_local_x)
```

Nota: eleLoad recibe primero la componente perpendicular (corte), luego la axial.

Paso 6: Retornar carga total

```
return q_horizontal * abs(dx) # Carga total en N para verificar equilibrio
```

4. NODOS, APOVOS Y ROTULA

Crear modelo

```
ops.wipe()
ops.model("basic", "-ndm", 2, "-ndf", 3)
```

wipe() limpia todo. *model()* crea un modelo nuevo 2D con 3 GDL/nodo.

Nodos

```
ops.node(1, 0.0, 0.0) # A - apoyo izquierdo
ops.node(2, 4.0, 3.0) # B - nodo intermedio
ops.node(3, 6.5, 3.0) # C - izquierda de rotula
ops.node(4, 6.5, 3.0) # C - derecha de rotula (MISMA posicion)
ops.node(5, 9.0, 3.0) # D - nodo intermedio
ops.node(6, 13.0, 0.0) # E - apoyo derecho
```

PREGUNTA CLAVE: Por que 2 nodos en C?

Respuesta: Para modelar la rotula interna. Los 2 nodos estan en la misma posicion pero tienen rotaciones independientes. Esto permite que no se transmita momento flector a traves de C.

Apovos (condiciones de frontera)

```
ops.fix(1, 1, 1, 0) # A: restringe ux=1, uy=1, rz=0 (articula)
ops.fix(6, 1, 1, 0) # E: restringe ux=1, uy=1, rz=0 (articula)
```

1 = restringido (fijo), 0 = libre.

Articula: no permite traslacion, pero si rotacion.

El marco tiene 2 articulas + 1 rotula interna = isostatico.

Rotula interna

```
ops.equalDOF(3, 4, 1, 2)
```

equalDOF(nodo_master, nodo_slave, DOF1, DOF2)

Iguala los DOF 1 (ux) y 2 (uy) entre los nodos 3 y 4.

El DOF 3 (rz) NO se iguala -> cada lado rota independiente.

Resultado: se transmite fuerza axial y corte, pero NO momento.

5. ELEMENTOS Y TRANSFORMACION

Transformacion geometrica

```
ops.geomTransf("Linear", 1)
```

Linear = sin considerar grandes deformaciones.

El tag 1 se reutiliza para todos los elementos (todos usan la misma transformacion).

Elementos viga-columna elasticos

```
ops.element('elasticBeamColumn', tag, nodo_i, nodo_j,
            area, E, I, geomTransf)

# Elemento 1: A-B
ops.element('elasticBeamColumn', 1, 1, 2, area, E, Iz, 1)
# Elemento 2: B-C (termina en nodo 3 = C izquierda)
ops.element('elasticBeamColumn', 2, 2, 3, area, E, Iz, 1)
# Elemento 3: C-D (empieza en nodo 4 = C derecha)
ops.element('elasticBeamColumn', 3, 4, 5, area, E, Iz, 1)
# Elemento 4: D-E
ops.element('elasticBeamColumn', 4, 5, 6, area, E, Iz, 1)
```

PREGUNTA CLAVE: Por que el elemento 3 empieza en nodo 4 y no en 3?

Respuesta: Porque en C hay rotula. El elemento BC termina en nodo 3 y el CD empieza en nodo 4. Al tener equalDOF, comparten desplazamientos pero no rotacion.

6. CARGAS Y ANALISIS

Definir patron de carga

```
ops.timeSeries("Linear", 1) # La carga se aplica en 1 paso
ops.pattern("Plain", 1, 1) # Patron de carga usando la serie 1
```

Aplicar carga distribuida en cada elemento

```
total_load = 0.0
total_load += add_vertical_load_on_horizontal_projection(1, 1, 2, q)
total_load += add_vertical_load_on_horizontal_projection(2, 2, 3, q)
total_load += add_vertical_load_on_horizontal_projection(3, 4, 5, q)
total_load += add_vertical_load_on_horizontal_projection(4, 5, 6, q)
```

Se acumula el total de carga para verificar equilibrio: $\sum(F_y) = 0$

Configurar el analisis

```
ops.system("BandGeneral") # Matriz de rigidez banda
ops.numberer("RCM") # Numeracion Reverse Cuthill-McKee
ops.constraints("Transformation") # Manejo de equalDOF
ops.integrator("LoadControl", 1.0) # 1 paso, factor 1.0
ops.algorithm("Linear") # Sin iteracion
ops.analysis("Static") # Estatico, no dinamico
```

Ejecutar

```
result = ops.analyze(1) # 1 paso de analisis
if result != 0: # Si falla...
    raise RuntimeError(...) # ...lanza error
```

*analyze(1) resuelve $K*u = F$. Si la matriz es singular o no converge, retorna $!= 0$.*

7. LEER RESULTADOS

Reacciones en apoyos

```
ops.reactions() # Calcula reacciones
```

```
R_Ax = ops.nodeReaction(1, 1) # Reaccion horizontal en A
```

```
R_Ay = ops.nodeReaction(1, 2) # Reaccion vertical en A
```

```
R_Ex = ops.nodeReaction(6, 1) # Reaccion horizontal en E
```

```
R_Ey = ops.nodeReaction(6, 2) # Reaccion vertical en E
```

nodeReaction(nodo, DOF)

DOF 1 = horizontal (ux), DOF 2 = vertical (uy), DOF 3 = momento (rz)

Fuerzas internas de cada elemento

```
local_forces = {ele: ops.eleResponse(ele, 'localForce')
                 for ele in range(1, 5)}
```

localForce retorna: [Ni, Vi, Mi, Nj, Vj, Mj]

N = axial, V = corte, M = momento

Subscript i = nodo inicial, j = nodo final

Encontrar maximos

Mayor fuerza axial

```
max_axial = max(abs(f[0]) for f in local_forces.values())
```

Mayor corte (compara ambos extremos de cada elemento)

```
max_shear = max(max(abs(f[1]), abs(f[4])) for f in local_forces.values())
```

Mayor momento

```
max_moment = max(max(abs(f[2]), abs(f[5])) for f in local_forces.values())
```

Calcular tensiones

```
sigma_flexion = max_moment / Wz # sigma = M/W (flexion)
```

```
sigma_axial = max_axial / A # sigma = N/A (axial)
```

Corte maximo (Jouravsky)

```
tau = max_shear * S/(I*t) # S/(I*t) = 0.0004028 1/mm2 de la pauta
```


8. VALIDACION CONTRA LA PAUTA

Valores de referencia (pauta)

```
R_Ay = R_Ey = 19.5 tonf
|R_Ax| = |R_Ex| = 21.13 tonf
|N|max = 28.6 tonf
|Q|max = 7.5 tonf
|M|max = 9.38 tonf*m
sigma flexion = 80.511 MPa
tau max = 29.6 MPa
```

Verificacion de equilibrio

```
vertical = R_Ay + R_Ey - total_load # Debe ser 0
horizontal = R_Ax + R_Ex # Debe ser 0
```

Assertions (validacion automatica)

```
# Equilibrio
assert isclose(horizontal_residual, 0, abs_tol=1e-7)
assert isclose(vertical_residual, 0, abs_tol=1e-7)

# Reacciones contra pauta
assert isclose(R_Ay_tonf, 19.5, abs_tol=1e-9)
assert isclose(R_Ey_tonf, 19.5, abs_tol=1e-9)
assert isclose(|R_Ax|, 21.13, abs_tol=0.01)
assert isclose(|R_Ex|, 21.13, abs_tol=0.01)

# Fuerzas internas contra pauta
assert isclose(|N|max, 28.6, abs_tol=0.1)
assert isclose(|Q|max, 7.5, abs_tol=0.1)
assert isclose(|M|max, 9.38, abs_tol=0.02)
```

*Si algun assert falla, el script aborta con AssertionError.
Esto garantiza que los resultados son correctos.*

Resultados esperados

```
R_Ax = 21.125 tonf
R_Ay = 19.500 tonf
R_Ex = -21.125 tonf
R_Ey = 19.500 tonf
|N|max = 28.600 tonf
|Q|max = 7.500 tonf
|M|max = 9.375 tonf*m
Estado: OK
```

9. PREGUNTAS QUE TE PUEDEN HACER

Es un wrapper de Python para OpenSees, un programa de analisis estructural desarrollado en UC Berkeley. Resuelve $K*u = F$ para estructuras lineales y no lineales.

2. Que significan ndm=2 y ndf=3?

ndm=2: modelo en 2 dimensiones (plano).

ndf=3: cada nodo tiene 3 grados de libertad: u_x (horizontal), u_y (vertical), r_z (rotacion fuera del plano).

3. Para que sirve equalDOF?

Iguala grados de libertad entre dos nodos. Se usa para modelar la rotula interna: los nodos comparten u_x y u_y , pero no r_z . Asi no se transmite momento a traves de C.

4. Que es elasticBeamColumn?

Un elemento viga-columna con comportamiento lineal elastico. Responde a fuerza axial, corte y momento flector. Requiere: area, modulo elastico E, inercia I, y transformacion.

5. Que es geomTransf Linear?

Define como se calculan las deformaciones del elemento. Linear =small strains, sin actualizar geometria. Es suficiente para analisis lineal elastico.

6. Que hace eleLoad?

Aplica una carga distribuida sobre un elemento. -beamUniform significa carga uniforme. Se ingresa en coordenadas LOCALES del elemento.

7. Por que la carga se convierte a locales?

El enunciado dice '3 tonf/m sobre proyeccion horizontal'. Pero los elementos estan inclinados. OpenSees recibe la carga en ejes locales, asi que hay que rotar la carga global a componentes $local_x$ (axial) y $local_y$ (perpendicular).

8. Que es localForce?

Retorna las fuerzas internas de un elemento en sus ejes locales:

[N_i , V_i , M_i , N_j , V_j , M_j]

N = axial, V = corte, M = momento en cada extremo (i y j).

9. Que es un marco isostatico?

Una estructura determinada estaticamente: se puede resolver con ecuaciones de equilibrio sin necesitar compatibilidad. Tiene 3 ecuaciones de equilibrio y 3 incognitas de reaccion (o equivalentes). La rotula interna reduce la indeterminacion.

10. Que es la verificacion?

Comparamos los resultados de OpenSees contra la pauta del profesor. Si coinciden dentro de tolerancia, el modelo es correcto. Las assertions automatizan esta verificacion.

10. COMANDOS UTILES

Ejecutar el script

```
python opensees/p1l0/ejemplo_minimo_2d.py
```

Ejecutar desde VS Code

Abrir terminal con Ctrl+` y pegar el comando anterior

Instalar dependencias

```
python -m pip install openseespy
```

```
python -m pip install matplotlib
```

```
python -m pip install fpdf2
```

Git:Actualizar repositorio

```
git pull # Bajar cambios del grupo
```

```
git add . # Seleccionar todos los cambios
```

```
git commit -m "msg" # Guardar cambios localmente
```

```
git push # Subir a GitHub
```

Ubicacion de archivos

```
opensees/p1l0/ejemplo_minimo_2d.py # El script
```

```
results/p1l0/diagrama_pregunta_2.png # El diagrama
```

```
docs/p1l0-explicacion.md # La explicacion
```

```
docs/p1l0-pregunta-2-control-1.md # Ejercicio base
```

```
AGENTS.md # Reglas para IA
```

Estructura del marco (recordar)

A(0,0) -> B(4,3) -> C(6.5,3) -> D(9,3) -> E(13,0)

```

  B _____ C _____ D
  /           |           \
  /           | (rotula)    \
  /           |           \
A              E

```

Apoyos: A = articula, E = articula

Carga: q = 3 tonf/m vertical